

人机交互技术中期报告

Cleaner: To-Do with AI — 中期报告

组员：袁野 10235101541; 陈磊 10235101567; 龙胜同 10235101479

一、项目内容

"Cleaner To-Do with AI"是一个使用 Flet 框架开发的桌面待办应用，核心特点是通过自然语言（中文优先）与 AI 交互来管理待办事项。

1. 核心功能

(1) AI 自然语言交互

用户可以通过聊天方式（例如输入"明天记得交报告"或"删除开会任务"）来创建、查询、修改、删除或标记任务。AI 意图识别由 LangChain 驱动，调用 DeepSeek API（OpenAI 兼容接口），将用户输入解析为结构化的 TaskPlan（Pydantic 模型），再路由到对应的业务操作。

(2) 本地数据存储

所有任务数据存储在本地 SQLite 数据库（`data/tasks.db`）中，无需联网即可使用基础功能。数据库在首次启动时自动创建，无需手动初始化。

(3) NLP 回退解析器

当 AI 服务不可用（未配置 API Key 或网络异常）时，系统自动回退到基于正则和关键词的解析器（`nlp_task_parser.py`），支持创建、删除、修改、完成/取消完成、查询等全部基础操作，保证离线可用性。

(4) 防误操作设计

- 删除操作需要二次确认：AI 返回待确认提示，用户点击"确认删除"按钮或输入"确认/是/yes"后才执行删除。

2. 技术架构

(1) UI 层（`ui/`）

基于 Flet 构建，采用可拖拽的双栏布局：左侧为 AI 对话面板，右侧为任务列表面板，分割条可用鼠标拖拽调整宽度。任务组件（`task_item.py`）支持行内编辑，并根据任务日期显示不同颜色标签（未来=蓝色，今天=橙色，过期=灰色）。全局使用 Microsoft YaHei 字体以保证中文显示效果。

(2) 服务层 (`services/`)

- `llm_service.py`: AI 编排核心。维护一个 LangChain ChatOpenAI 实例 (planner, temperature=0.3) 用于意图规划, 另一个实例 (chat_model, temperature=0.4) 用于自由对话。通过 UUID token 机制管理待确认的删除操作。
- `task_service.py`: 业务逻辑层。封装任务增删改查、批量操作、状态切换, 以及自然日期解析 (今天/明天/后天/ISO 格式/MM-DD 格式)。
- `nlp_task_parser.py`: 正则回退解析器。通过关键词匹配和多步文本清洗提取任务意图, 支持批量数量识别、日期提取、核心任务名提取。

(3) 数据层 (`storage/` 与 `core/`)

- `task_repo.py`: 封装所有 SQLite CRUD 操作, 包括单条/批量创建、查询、模糊搜索、更新、删除。
- `task.py`: 定义 TaskRecord 数据模型 (id、name、date、completed), 提供 `from_row()` 和 `to_db_values()` 用于数据库与模型的互转。
- `db.py`: 管理数据库连接, 自动创建数据目录和 tasks 表 (幂等)。

3. 运行与配置

项目通过 `python app.py` 启动。需在项目根目录创建 `.env` 文件并填入 API 密钥, 才能启用完整的 AI 对话功能:

```
OPENAI_API_KEY=<DeepSeek API Key>

OPENAI_MODEL=deepseek-chat

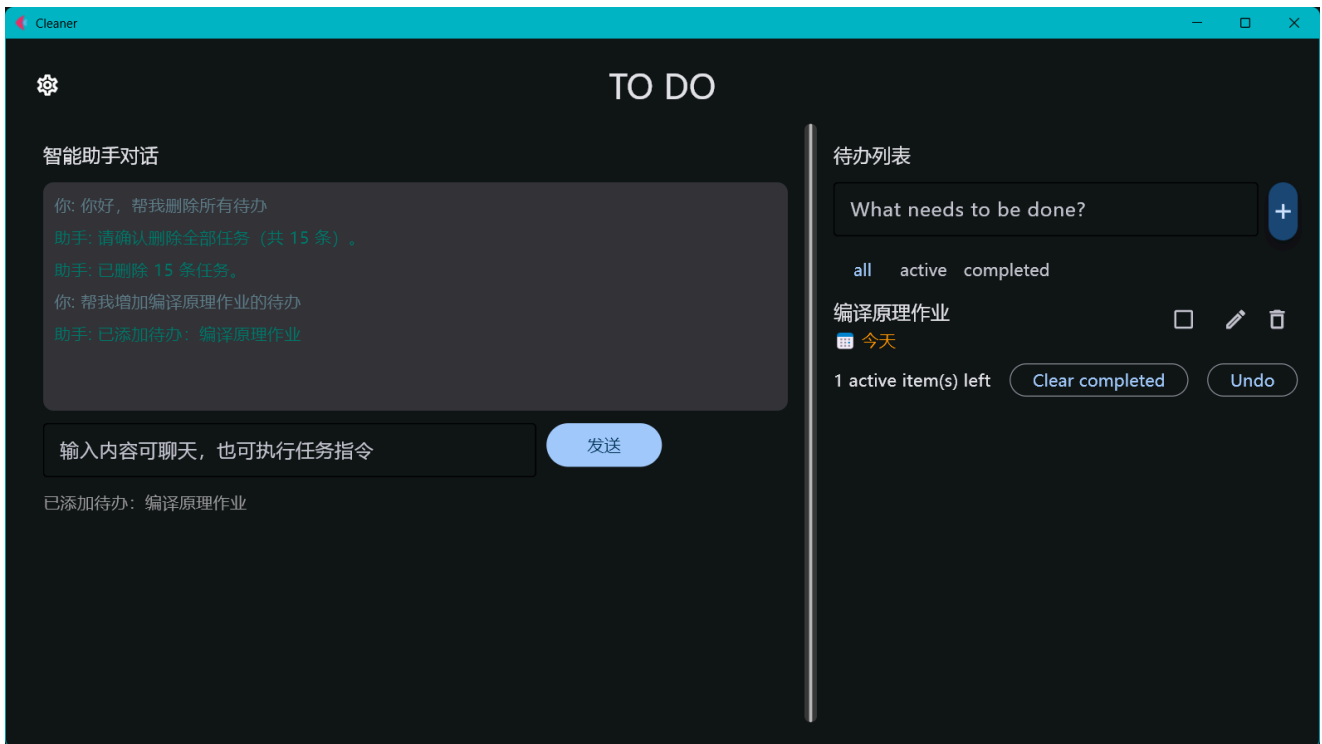
OPENAI_BASE_URL=https://api.deepseek.com/

OPENAI_TEMPERATURE=0.3
```

不配置 API Key 时, AI 对话不可用, 但手动任务操作和 NLP 回退解析器仍正常运行。

二、当前进展

本项目已实现一个功能完整的可用版本, 核心模块均已开发完毕并可交互运行。



1. 已完成模块

(1) 完整的 AI 任务操作闭环

从用户输入 → LLM 意图规划 (TaskPlan Pydantic 模型) → 业务逻辑执行 (TaskService) → 数据持久化 (SQLite) 的完整流程已全部打通, 支持创建、查询、修改、完成/取消完成、删除等全部操作类型。

(2) 删除确认与撤销机制

删除二次确认 (UUID token 机制) 和 20 步撤销 (任务列表快照) 均已实现并集成到 UI 交互流程中。

(3) 完整的用户界面

主视图 (`todo_view.py`)、任务行组件 (`task_item.py`)、双栏拖拽布局、任务状态筛选 (全部/未完成/已完成)、日期颜色编码、主题字体设置均已实现。

(4) NLP 回退解析器

`nlp_task_parser.py` 已完整实现, 覆盖全部操作类型, 在 AI 服务不可用时自动接管解析工作。

(5) AI 自由对话

当用户输入不属于任务操作时, 系统自动切换到自由对话模式 (chat 方法), 给出友好的中文回复。

2. 项目结构现状

app.py	入口
core/	
constants/enums.py	TaskActionType 枚举
models/task.py	TaskRecord 数据模型
storage/	
db.py	SQLite 连接与建表
task_repo.py	CRUD 仓储层
services/	
task_service.py	业务逻辑、日期解析
llm_service.py	LangChain 编排、意图规划
nlp_task_parser.py	正则回退解析器
ui/	
theme.py	颜色常量、全局主题
views/todo_view.py	主视图 (TodoApp)
components/task_item.py	单任务行组件
data/tasks.db	本地数据库 (自动创建)

三、下一步计划

1. 开发预留功能模块

(1) 设置系统

开发 `settings_view.py` 和相关组件，允许用户在界面内配置 API Key、模型选择、主题等应用行为，替代当前手动编辑 `.env` 的方式。

(2) 日历视图

开发 `calendar_service.py` 和 `calendar_view.py`，提供任务的时间线或日历视图，方便用户按日期维度管理任务。

(3) 国际化 (i18n)

利用预留的 `i18n.py` 框架增加对多语言的支持，当前界面混用中英文的部分（如"Clear completed"、"What needs to be done?"）统一为可配置文案。

(4) 应用状态管理

完善 `state.py`，将当前分散在 TodoApp 中的状态（筛选状态、待确认 token、undo 栈等）集中管理，提升可维护性。

2. 优化与增强

(1) UI/UX 优化

- 将右侧面板的英文占位文案（"What needs to be done?"、"Clear completed"等）替换为中文，与整体风格保持一致。
- 优化聊天历史区域高度，支持更多消息展示。

(2) AI 提示词迭代

根据实际使用中出现的意图识别偏差，持续优化 `llm_service.py` 中的系统提示词和 few-shot 示例。

(3) 新功能探索

可能增加任务分类标签、优先级标记、到期提醒通知等功能，根据后续用户反馈决定优先级。

四、人机交互设计分析

本项目作为人机交互课程作业，在设计与实现中贯穿了多项 HCI 核心原则。以下从交互模式、反馈机制、容错设计、视觉设计四个维度展开分析。

1. 交互模式

(1) 自然语言交互 (NLI)

左侧 AI 对话面板是本项目最核心的交互创新。用户无需学习特定命令语法，直接用日常中文描述意图（如"明天记得交报告"、"把开会任务改成后天"），系统通过 LangChain +

DeepSeek API 将自然语言解析为结构化的 `TaskPlan` (Pydantic 模型)，再路由到对应业务操作。这一设计降低了认知负担，符合 HCI 中"以用户为中心"的设计原则。

当 AI 服务不可用时，系统自动切换到基于正则和关键词的 NLP 回退解析器 (`nlp_task_parser.py`)，保证离线场景下的基础可用性，体现了渐进增强的交互设计思路。

(2) 直接操作 (Direct Manipulation)

右侧任务列表面板支持传统的直接操作交互：

- 复选框一键切换任务完成状态
- 行内编辑：点击编辑图标后，任务名称原地变为可编辑文本框，修改后点击确认图标提交
- 状态筛选按钮（全部 / 未完成 / 已完成）即时过滤任务列表
- 可拖拽分割条：用鼠标拖动左右面板之间的分割条，实时调整两侧面板宽度

两种交互模式并存，用户可根据习惯自由选择，兼顾了不同用户群体的使用偏好。

2. 反馈机制

(1) 即时视觉反馈

- 对话气泡颜色区分：用户消息与 AI 回复使用不同背景色。
- 日期颜色编码：任务日期标签根据时间状态动态着色——未来任务显示蓝色，今天到期显示橙色，已过期显示灰色。这一设计让用户无需阅读日期文字即可感知任务紧迫程度。
- 动态任务计数：右侧面板底部实时显示当前未完成任务数量，帮助用户随时掌握整体进度。
- 筛选按钮高亮：当前激活的筛选状态对应按钮颜色加深，明确指示系统当前所处状态。
- 聊天记录自动滚动：每次新消息出现后，聊天区域自动滚动到底部，避免用户手动翻找最新内容。

(2) 操作结果反馈

AI 处理完用户指令后，在对话区域以自然语言回复操作结果（如"已创建任务：明天交报告"），同时右侧任务列表立即刷新，形成"文字确认 + 视觉更新"的双重反馈闭环。

3. 视觉与可访问性设计

(1) 中文优先的字体方案

全局统一使用 Microsoft YaHei 字体 (`theme.py:get_app_theme()`)，确保中文字符在 Windows 平台上的清晰显示，避免系统默认字体导致的中文渲染问题。

(2) 语义化图标与工具提示

任务行的编辑、删除、确认按钮均配有 `tooltip` 属性，鼠标悬停时显示操作说明，降低图标歧义，提升可发现性 (Discoverability)。

(3) 响应式布局

主视图采用 `ResponsiveRow` 配合 `xs/md` 断点，在不同窗口尺寸下自动调整列宽，保证界面在小窗口和大屏下均可正常使用。

(4) 集中式主题管理

所有颜色常量集中定义在 `AppColors`，语义化命名，便于统一调整视觉风格，保证界面色彩的一致性。